

Code Review: TeleConnect AI Platform

Reviewer: Delivery Lead, AI Platform

Date: June 2026

Scope: model_pipeline.py, agent.py, tools.py, eval_suite.py, config.py

This is a review of the full codebase as inherited. The short version: the system has five issues that would cause immediate failures if deployed as-is, and several more that would quietly corrupt results over time. I've grouped them by severity and file, followed by a plan for what to fix first and why.

Issue Identification

config.py

ID	Issue	Severity	Why It Matters	Fix
C-1	OPENAI_API_KEY = "sk-proj-abc123..." hardcoded in source	Critical	This key is in version control. Every person with repo access has it. Rotate it immediately, then load from environment variable: os.environ["OPENAI_API_KEY"]. Add .env to .gitignore.	Environment variable + immediate key rotation
C-2	MAX_RETRIES = 0 defined but nothing uses it	High	The config implies retry behaviour exists. It doesn't. One transient API error and the agent crashes mid-session with no recovery.	Import in agent.py, wrap completions calls with tenacity exponential backoff, minimum 3 retries
C-3	agent.py ignores config values, hardcodes "gpt-4o" directly	Medium	Central config is pointless if modules don't use it. Changes to model or temperature in config have no effect.	Import LLM_MODEL and LLM_TEMPERATURE from config everywhere they're used

model_pipeline.py

ID	Issue	Severity	Why It Matters	Fix
M-1	<code>churn_rate_by_contract</code> is computed on the full dataset before the train/test split	High	Data leakage. The test set's churn labels influence the feature values used in training, which inflates reported AUC. The model looks better than it is.	Compute this feature using only training data, then apply the mapping to the test set after the split
M-2	<code>LabelEncoder</code> instances are fit and immediately thrown away never saved	Critical	At inference time, <code>predict_churn</code> receives raw strings like "annual" for <code>contract_type</code> , but the model expects integers. Either a <code>ValueError</code> on the first real prediction, or silent garbage output.	Save encoders alongside the model in a <code>ModelArtifact</code> bundle (joblib). Apply them at serving time before prediction
M-3	<code>predict_churn</code> expects engineered features (<code>charge_per_tenure</code> , <code>churn_rate_by_contract</code> , etc.) that raw customer records don't have	Critical	<code>lookup_customer</code> returns raw CSV fields. Calling <code>predict_churn</code> on that result immediately <code>KeyErrors</code> on every missing derived column. The agent's core flow lookup then score is broken from end to end.	Feature engineering must be part of the serving pipeline. The <code>predict_churn</code> function should accept a raw customer dict and internally apply all transformations before calling the model
M-4	Model is reloaded from disk on every single <code>predict_churn</code> call	High	<code>pickle.load()</code> on every request adds 100-500ms per prediction. Under any real load this blows the 5-second response SLA.	Load model once at startup. A module-level <code>_model = None</code> with lazy init is fine; loading per request is not
M-5	Model serialised with <code>pickle</code>	High	<code>pickle.load()</code> will execute arbitrary code. A corrupted or replaced model file is a remote code execution risk. It also breaks across Python versions.	Use <code>joblib</code> instead. Add a SHA-256 checksum check before loading
M-6	<code>interaction_recency</code> is computed against a hardcoded "2024-07-01" reference date	Medium	This feature gets further from reality every day without retraining. A customer last contacted in June 2024 was 1 day ago at training time; they're over 700 days ago now. The feature silently drifts.	Use <code>pd.Timestamp.now()</code> at feature engineering time. Document in the model card that this feature is relative to training date and set a retraining schedule
M-7	<code>train_test_split</code> has no <code>stratify=y</code>	Medium	Churn datasets are imbalanced. An unstratified split can produce a test set with a meaningfully different churn rate, making all evaluation metrics unreliable.	Add <code>stratify=y</code> to the split call
M-8	<code>roc_auc_score</code> is called with binary predictions, not probabilities	Medium	<code>model.predict()</code> returns 0/1 labels. Passing these to <code>roc_auc_score</code> computes something close to accuracy, not AUC. The number printed at training time is wrong.	Use <code>model.predict_proba(X_test)[: , 1]</code> instead

ID	Issue	Severity	Why It Matters	Fix
M-9	<code>fillna(df.mean())</code> is computed on the full dataset including test rows	Low	Minor leakage imputation stats should come from training data only. Effect is small but the principle is wrong.	Split first, then compute <code>fill_values = X_train.mean()</code> and apply to both sets

agent.py

ID	Issue	Severity	Why It Matters	Fix
A-1	<code>conversation_history = []</code> is a module-level global shared across all calls	Critical	Every session for every user appends to the same list. Customer A's data ends up in Customer B's context. Memory grows without bound until the process crashes. This is both a data privacy breach and a correctness bug.	Scope history to each session. Simplest fix: pass it as a parameter <code>def run_agent(message, history=None)</code> and initialise fresh per call. Add a max-turn limit.
A-2	No timeout on any API call	Critical	If the API hangs, <code>run_agent</code> hangs with it forever. With a 5-second SLA, one stuck request blocks a rep indefinitely.	Add <code>timeout=4.0</code> to every <code>client.chat.completions.create()</code> call
A-3	<code>MAX_RETRIES</code> from config is never imported or applied	High	Dead config. Transient errors (rate limits, 502s) raise unhandled exceptions and crash the session.	Import and wire up with exponential backoff see C-2
A-4	Tool-call loop has no iteration limit	High	A confused model can call tools indefinitely. There is no stop condition beyond the model choosing to stop.	Add <code>MAX_TOOL_ITERATIONS = 10</code> and break with an error if exceeded
A-5	Tool errors like <code>{"error": "Customer not found"}</code> are passed straight back to the model	High	The model doesn't know it received an error it just sees unexpected content and hallucinates around it. A rep gets fabricated customer details for a customer that doesn't exist.	Check for "error" key in tool results. Return a structured message to the rep and stop, rather than continuing.
A-6	<code>temperature=0.7</code> for tool-calling	Medium	High temperature is appropriate for creative tasks. Tool selection should be deterministic. At 0.7, the same query produces different tool sequences on different runs.	Use <code>temperature=0.1</code> for tool-calling turns; reserve higher values for the final recommendation text

ID	Issue	Severity	Why It Matters	Fix
A-7	ESCALATION_KEYWORDS is defined in config but never imported or checked	Medium	The system prompt tells the model to escalate when it detects words like "lawyer" or "cancel". There is no code that actually does this.	Import the list, check user_message against it, and call escalate_to_supervisor when triggered but first fix T-8
A-8	execute_tool returns None for unrecognised tool names	Medium	If the model calls a tool that doesn't exist, None gets JSON-serialised as null and sent back to the model. This causes confusing follow-up behaviour.	Return {"error": f"Unknown tool: {name}"} and log it

tools.py

ID	Issue	Severity	Why It Matters	Fix
T-1	CUSTOMER_DATA = pd.read_csv(...) runs at import time	Critical	If the CSV is missing, the module fails to import. Nothing starts. No useful error message.	Lazy-load on first use. Wrap in a try/except with a clear RuntimeError message.
T-2	predict_churn is duplicated from model_pipeline.py	High	Two independent copies of the same logic. When one is fixed, the other isn't. They will diverge.	Delete the copy in tools.py. Import and call the one in model_pipeline.
T-3	lookup_customer returns the entire raw CSV row to the LLM	High	Full customer record including fields that have nothing to do with retention goes into the prompt. More tokens, more distraction, more surface area for PII exposure.	Define an allowlist of fields relevant to retention decisions (tenure, contract type, charges, satisfaction score) and filter before returning
T-4	predict_churn in tools.py expects engineered features; lookup_customer returns raw fields	Critical	Same root cause as M-3. Calling these two tools in sequence the most natural thing for the agent to do KeyErrors on every missing derived feature.	Fix is the same: make predict_churn accept raw customer data and handle feature engineering internally
T-5	Model loaded from disk on every prediction	High	Same as M-4. Load it once.	See M-4
T-6	interaction_log = [] in memory only	High	Every logged interaction is lost on restart. In a regulated environment, this is not a log it's a buffer that gets discarded.	Write to PostgreSQL or SQLite. Include timestamp, session ID, and model version in every row.

ID	Issue	Severity	Why It Matters	Fix
T-7	<code>get_retention_offers</code> ignores its <code>risk_tier</code> and <code>contract_type</code> arguments entirely	High	Every customer gets every offer regardless of their risk profile. The entire point of risk stratification is defeated. A low-risk customer on a two-year contract is being offered the same 30% discount as a churning month-to-month customer.	Actually filter by tier and contract type. The offer catalogue and tier mapping are small enough to implement in a few lines.
T-8	<code>escalate_to_supervisor</code> is defined but not registered in the TOOLS list	Medium	The model cannot call a tool it doesn't know about. The escalation path the system prompt promises does not exist.	Add the function definition to <code>TOOLS</code> in <code>agent.py</code>

eval_suite.py

ID	Issue	Severity	Why It Matters	Fix
E-1	<code>evaluate_tool_accuracy</code> always returns 1.0	Critical	The function never checks which tools were actually called. It returns 1.0 any time <code>expected_tools</code> is non-empty. Every eval run reports 100% tool accuracy no matter what the agent did. This is the most consequential bug in the file the metric everyone is relying on to validate agent behaviour is completely fabricated.	Instrument <code>execute_tool</code> to record which functions were called. Compare that set against <code>expected_tools</code> : <code>len(expected & actual) / len(expected)</code> .
E-2	Test cases share global <code>conversation_history</code>	Critical	TC-01's conversation is still in history when TC-02 runs. Results depend on test order. None of the eval results are trustworthy as individual test scores.	Clear history before each test case. <code>conversation_history.clear()</code> or refactor <code>run_agent</code> to take history as a parameter.
E-3	<code>evaluate_response_completeness</code> matches keywords from the description string, not the response	Medium	The description says "should include churn probability and risk tier". The function checks whether words like "should", "include", "churn" appear in the agent response. This is not a meaningful evaluation.	Check for the specific values that should appear the actual churn probability, the actual risk tier using regex or structured parsing
E-4	LLM judge runs at <code>temperature=0.7</code>	Medium	The same test case produces different scores on every run. You cannot detect regressions with a non-deterministic judge.	Set <code>temperature=0</code> . If scores feel too mechanical, run three passes and take the median.

ID	Issue	Severity	Why It Matters	Fix
E-5	Eval runs against live agent with real customer IDs	Medium	Eval traffic hits production tools. <code>log_interaction</code> entries from test runs pollute the production audit log.	Use synthetic fixture customers (IDs like TEST-001) stored in version control. Point the eval at a separate data store.
E-6	Five test cases, all happy path	Medium	No coverage of: customer not found, tool error, escalation trigger, empty offer result, adversarial input. The eval has never been asked a hard question.	Extend to at least 15 cases. Include at least one failure case per tool.
E-7	<code>eval_results.json</code> is overwritten on every run	Medium	No history. Cannot compare this run to the last run. Regression detection is impossible.	Append a timestamp and model version to the filename. Keep a result index.
E-8	LLM judge has no calibration examples	Low	Without anchors, the judge's scoring scale is arbitrary and inconsistent across runs and model versions.	Add two or three labelled examples to the judge prompt showing what a score of 1, 3, and 5 actually looks like

Remediation Plan

Two sprints before the on-prem deployment. The migration makes these bugs worse, not easier to live with a latency issue that occasionally causes a slow response on cloud becomes a hard blocker on local GPU hardware, and data leakage that was a code smell becomes a compliance violation in a regulated environment. So the ordering below is deliberate.

Sprint 1 must be done before anything is deployed

The five critical issues block deployment outright. Fix them first, in this order:

- **C-1** Rotate the API key and move it to an environment variable. This takes an hour and needs to happen today, independent of everything else.
- **A-1** Global conversation history. Customer data is leaking between sessions right now in the cloud deployment. This is a privacy issue, not just a code issue.
- **A-2** Add a 4-second timeout to every API call. Without this, the 5-second SLA is physically impossible to guarantee.
- **E-1 and E-2** Fix the eval suite. The tool accuracy metric always returning 100% means we currently have no way to measure whether the agent is working. We need this fixed before we can validate anything else.

Once those are done, the high-severity items that block on-prem specifically:

- **M-2, M-3, T-4** The serving pipeline is broken. Build a `ModelArtifact` that bundles the trained model with its fitted encoders and feature engineering logic. `predict_churn` must accept raw customer data, not pre-engineered features. This is the most work in Sprint 1 estimate 6 hours.
- **M-4, T-5** Load the model once at startup. Disk reads on every prediction will kill the 5-second SLA.

- **T-1** Move customer data loading out of module import scope.
- **T-6** Wire interaction logging to a real database. An in-memory list is not an audit trail.
- **T-7** Implement offer filtering in `get_retention_offers`. This function is the whole point of risk stratification.
- **A-4** Add a tool-call iteration limit. Without it, a confused model can loop indefinitely.

Sprint 2 before the next model retrain

These don't block launch but will cause problems during the model's first retraining cycle:

- M-1: Fix `churn_rate_by_contract` data leakage. The current AUC is meaningfully optimistic.
- M-7, M-8: Stratified split, AUC using probabilities. Basic evaluation hygiene.
- M-5: Replace pickle with joblib. Before the model is updated in the air-gapped environment.
- M-6: Dynamic reference date for `interaction_recency`.
- A-7, T-8: Wire up escalation. The system prompt promises it.
- E-3 through E-7: Improve eval quality and coverage. The eval suite needs to be trustworthy before it can catch regressions.

Track as debt, address when the opportunity arises

`ESCALATION_KEYWORDS` in config is dead code (C-2 prerequisite). Minor fillna leakage (M-9). LLM judge calibration examples (E-8). The offer catalogue lives in source code and belongs in a database.

Process Observation

What strikes me about this codebase isn't any individual bug it's that the most important metric in the eval suite always returns 100% and nobody caught it. That's a signal about process, not just code. The system was tested by running it, not by verifying that the tests themselves measure what they claim to. `evaluate_tool_accuracy` was presumably written, run, seen to produce good-looking numbers, and trusted. There was no test of the test.

The same pattern shows up in the serving pipeline. The feature engineering code in training and the feature expectations in `predict_churn` have drifted apart they never shared a contract. Someone added a derived feature to training, forgot to add it to serving, and the mismatch sat there untouched because there was no integration test that called `predict_churn` with real customer data all the way through.

Three things would prevent this class of issue from recurring: first, a CI gate that runs the eval suite on every pull request against synthetic fixture data which forces `evaluate_tool_accuracy` to be correct or the pipeline fails. Second, a model promotion gate that calls `predict_churn` with a raw customer record as part of validating any new model artifact which catches the serving/training gap immediately. Third, structured logging on the agent from the first day it handles real traffic tool calls, outcomes, latencies so that broken behaviour like always returning the same offers or never triggering escalation shows up in dashboards before a user reports it.

The team built something real here. The architecture is sensible, the tool structure is right, and the evaluation approach is sound in principle. The issues are fixable in two sprints. But the process gap building without feedback loops will generate the same class of bugs in the next feature if it isn't addressed.